

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

РУБЕЖНЫЙ КОНТРОЛЬ 1

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-31Б _____ Смирнов А.Д.
(Подпись) (Ф.И.О.)

Проверил: _____ Гапанюк Ю.Е.
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:

- Оценка:

2026

Вариант запросов: А

Предметная область: №26 — Студенческая группа и Учебный курс

Классы и связи

Класс Student хранит идентификатор студента, его ФИО, количественный признак (средний балл) и идентификатор основного учебного курса. Класс Course хранит идентификатор и название учебного курса. Класс StudentCourse содержит пары идентификаторов для связи многие-ко-многим.

Связь один-ко-многим задаётся полем course_id в объекте Student: у студента один основной курс, а на курсе может находиться несколько студентов. Для связи многие-ко-многим используется список объектов StudentCourse, поэтому один студент может быть записан на несколько курсов (и наоборот).

Тестовые данные

Используются три учебных курса, пять студентов и десять записей связи. Идентификаторы в связях соответствуют существующим объектам. Несколько студентов связаны более чем с одним курсом, что позволяет проверить запрос А3 (многие-ко-многим).

Исходный код программы:

```
from operator import itemgetter


class Student:
    """Студент (Связь: много)."""
    def __init__(self, id, fio, score, course_id):
        self.id = id
        self.fio = fio
        self.score = score      # Количественный признак
(средний балл)
        self.course_id = course_id


class Course:
```

```
    """Учебный курс (Связь: один)."""
    def __init__(self, id, name):
        self.id = id
        self.name = name

class StudentCourse:
    """Связь многие-ко-многим: Студент - Курс."""
    def __init__(self, student_id, course_id):
        self.student_id = student_id
        self.course_id = course_id

# Тестовые данные
courses = [
    Course(1, 'Базовый курс программирования'),
    Course(2, 'Математический анализ'),
    Course(3, 'Физика'),
    Course(11, 'Продвинутый курс Python'),
    Course(22, 'Дискретная математика'),
    Course(33, 'Квантовая физика')
]

students = [
    Student(1, 'Артамонов', 4.5, 1),
    Student(2, 'Петров', 3.8, 2),
    Student(3, 'Иваненко', 5.0, 3),
    Student(4, 'Иванов', 4.2, 3),
    Student(5, 'Иванин', 3.5, 3)
]

students_courses = [
    StudentCourse(1, 1), StudentCourse(1, 11),
    StudentCourse(2, 2), StudentCourse(2, 22),
    StudentCourse(3, 3), StudentCourse(3, 33),
    StudentCourse(4, 3), StudentCourse(4, 33),
```

```
    StudentCourse(5, 3), StudentCourse(5, 33)
]

def main():
    # Соединение данных один-ко-многим
    one_to_many = [(s.fio, s.score, c.name)
                   for c in courses
                   for s in students
                   if s.course_id == c.id]

    # Соединение данных многие-ко-многим
    many_to_many_temp = [(c.name, sc.course_id, sc.student_id)
                         for c in courses
                         for sc in students_courses
                         if c.id == sc.course_id]

    many_to_many = [(s.fio, s.score, course_name)
                    for course_name, course_id, student_id in
many_to_many_temp
                    for s in students if s.id == student_id]

    print('Задание А1')
    # Сортировка по названию курса (индекс 2)
    res_11 = sorted(one_to_many, key=itemgetter(2))
    for item in res_11:
        print(item)

    print('\nЗадание А2')
    res_12_unsorted = []
    # Перебираем все курсы
    for c in courses:
        c_students = list(filter(lambda i: i[2] == c.name,
one_to_many))
        if len(c_students) > 0:
```

```
            c_scores = [score for _, score, _ in c_students]
            c_scores_sum = sum(c_scores) # Суммарный балл
            res_12_unsorted.append((c.name, c_scores_sum))

    # Сортировка по суммарному баллу (по убыванию)
    res_12 = sorted(res_12_unsorted, key=itemgetter(1),
reverse=True)
    for item in res_12:
        print(item)


    print('\nЗадание А3')
    res_13 = {}
    # Перебираем все курсы
    for c in courses:
        if 'курс' in c.name.lower():
            c_students = list(filter(lambda i: i[2] == c.name,
many_to_many))
            c_students_names = [x for x, _, _ in c_students]
            res_13[c.name] = c_students_names


    for key, value in res_13.items():
        print(f"'{key}': {value}")

if __name__ == '__main__':
    main()
```

Результаты выполнения программы

```
andre@DESKTOP-RP8GFFD:~/pl-paradigms-labs$ python3 rk1_variant_A26.py
Задание A1
('Артамонов', 4.5, 'Базовый курс программирования')
('Петров', 3.8, 'Математический анализ')
('Иваненко', 5.0, 'Физика')
('Иванов', 4.2, 'Физика')
('Иванин', 3.5, 'Физика')

Задание A2
('Физика', 12.7)
('Базовый курс программирования', 4.5)
('Математический анализ', 3.8)

Задание A3
'Базовый курс программирования': ['Артамонов']
'Продвинутый курс Python': ['Артамонов']
```